

NAME

RDKitFilterPAINS.py - Filter PAINS molecules

SYNOPSIS

```
RDKitFilterPAINS.py [--infileParams <Name,Value,...>] [--mode <filter or count>] [--mp <yes or no>] [
--mpParams <Name,Value,...>] [--outfileFiltered <yes or no>] [ --outfileParams <Name,Value,...> ] [
--painsMode <All or A, B, C>] [--painsMatch <First or All>] [--negate <yes or no>] [--overwrite] [-w <dir>]
-i <infile> -o <outfile>
```

RDKitFilterPAINS.py -h | --help | -e | --examples

DESCRIPTION

Filter Pan-assay Interference molecules (PAINS) [Ref 130 - 131] from an input file by performing a substructure search using SMARTS pattern specified in MAYACHEMTOOLS/lib/data/PAINFilters.csv file and write out appropriate molecules to an output file or simply count the number of filtered molecules.

The supported input file formats are: SD (.sdf, .sd), SMILES (.smi, .csv, .tsv, .txt)

The supported output file formats are: SD (.sdf, .sd), SMILES (.smi)

OPTIONS

-e, --examples

Print examples.

-h, --help

Print this help message.

-i, --infile <infile>

Input file name.

--infileParams <Name,Value,...> [default: auto]

A comma delimited list of parameter name and value pairs for reading molecules from files. The supported parameter names for different file formats, along with their default values, are shown below:

```
SD: removeHydrogens,yes,sanitize,yes,strictParsing,yes
SMILES: smilesColumn,1,smilesNameColumn,2,smilesDelimiter,space,
        smilesTitleLine,auto,sanitize,yes
```

Possible values for smilesDelimiter: space, comma or tab.

-m, --mode <filter or count> [default: filter]

Specify whether to filter the matched molecules and write out the rest of the molecules to an outfile or simply count the number of matched molecules marked for filtering.

--mp <yes or no> [default: no]

Use multiprocessing.

By default, input data is retrieved in a lazy manner via mp.Pool.imap() function employing lazy RDKit data iterable. This allows processing of arbitrary large data sets without any additional requirements memory.

All input data may be optionally loaded into memory by mp.Pool.map() before starting worker processes in a process pool by setting the value of 'inputDataMode' to 'InMemory' in '--mpParams' option.

A word to the wise: The default 'chunkSize' value of 1 during 'Lazy' input data mode may adversely impact the performance. The '--mpParams' section provides additional information to tune the value of 'chunkSize'.

--mpParams <Name,Value,...> [default: auto]

A comma delimited list of parameter name and value pairs to configure multiprocessing.

The supported parameter names along with their default and possible values are shown below:

```
chunkSize, auto
inputDataMode, Lazy [ Possible values: InMemory or Lazy ]
numProcesses, auto [ Default: mp.cpu_count() ]
```

These parameters are used by the following functions to configure and control the behavior of multiprocessing: `mp.Pool()`, `mp.Pool.map()`, and `mp.Pool.imap()`.

The `chunkSize` determines chunks of input data passed to each worker process in a process pool by `mp.Pool.map()` and `mp.Pool.imap()` functions. The default value of `chunkSize` is dependent on the value of 'inputDataMode'.

The `mp.Pool.map()` function, invoked during 'InMemory' input data mode, automatically converts RDKit data iterable into a list, loads all data into memory, and calculates the default `chunkSize` using the following method as shown in its code:

```
chunkSize, extra = divmod(len(dataIterable), len(numProcesses) * 4)
if extra: chunkSize += 1
```

For example, the default `chunkSize` will be 7 for a pool of 4 worker processes and 100 data items.

The `mp.Pool.imap()` function, invoked during 'Lazy' input data mode, employs 'lazy' RDKit data iterable to retrieve data as needed, without loading all the data into memory. Consequently, the size of input data is not known a priori. It's not possible to estimate an optimal value for the `chunkSize`. The default `chunkSize` is set to 1.

The default value for the `chunkSize` during 'Lazy' data mode may adversely impact the performance due to the overhead associated with exchanging small chunks of data. It is generally a good idea to explicitly set `chunkSize` to a larger value during 'Lazy' input data mode, based on the size of your input data and number of processes in the process pool.

The `mp.Pool.map()` function waits for all worker processes to process all the data and return the results. The `mp.Pool.imap()` function, however, returns the results obtained from worker processes as soon as the results become available for specified chunks of data.

The order of data in the results returned by both `mp.Pool.map()` and `mp.Pool.imap()` functions always corresponds to the input data.

`-n, --negate <yes or no> [default: no]`

Specify whether to filter molecules not matching the PAINS filters specified by SMARTS patterns.

`-o, --outfile <outfile>`

Output file name.

`--outfileFiltered <yes or no> [default: no]`

Write out a file containing filtered molecules. Its name is automatically generated from the specified output file. Default: `<OutfileRoot>_Filtered.<OutfileExt>`.

`--outfileParams <Name,Value,...> [default: auto]`

A comma delimited list of parameter name and value pairs for writing molecules to files. The supported parameter names for different file formats, along with their default values, are shown below:

```
SD: compute2DCoords,auto,kekulize,yes,forceV3000,no
SMILES: smilesKekulize,no,smilesDelimiter,space, smilesIsomeric,yes,
        smilesTitleLine,yes,smilesMolName,yes,smilesMolProps,yes
```

Default value for `compute2DCoords`: yes for SMILES input file; no for all other file types.

`--overwrite`

Overwrite existing files.

`-p, --painsMode <All or A, B, or C> [default: All]`

All or a comma delimited list of PAINS filter family type to used for filtering molecules.

`--painsMatch <First or All> [default: First]`

Stop after matching only first PAINS pattern or match all patterns for filtering molecules.

The 'PAINSAAlertCount' and 'PAINSAAlerts' data fields are added to SD file containing filtered molecules for 'All' value of '-painsMatch'. In addition, these data fields are only written to tab or comma delimited SMILES file.

Format:

```
> <PAINSAAlertsCount>
```

Number

```
> <PAINSAalerts>
FilterType: ID; FilterType: ID... ..''
```

-w, --workingdir <dir>

Location of working directory which defaults to the current directory.

EXAMPLES

To count the number of molecules not containing any substructure corresponding to PAINS SMARTS patterns and write out a SMILES file, type:

```
% RDKitFilterPAINS.py -i Sample.smi -o SampleOut.smi
```

To count the number of molecules not containing any substructure corresponding to PAINS SMARTS patterns and write out a SMILES file containing these and filtered molecules along with the alerts information for filtered molecules matching first pattern, type:

```
% RDKitFilterPAINS.py --outfileFiltered yes --outfileParams
"SMILESDelimiter,comma" -i Sample.smi -o SampleOut.smi
```

To count the number of molecules not containing any substructure corresponding to PAINS SMARTS patterns and write out comma delimited SMILES files containing these and filtered molecules along with the alerts information for filtered molecules matching all patterns, type:

```
% RDKitFilterPAINS.py --painsMatch All --outfileFiltered yes
--outfileParams "SMILESDelimiter,comma" -i Sample.sdf
-o SampleOut.smi
```

To count the number of molecules not containing any substructure corresponding to PAINS SMARTS patterns and write out comma delimited SD files containing these and filtered molecules along with the alerts information for filtered molecules matching all patterns, type:

```
% RDKitFilterPAINS.py --painsMatch All --outfileFiltered yes
-i Sample.smi -o SampleOut.sdf
```

To count the number of molecules not containing any substructure corresponding to PAINS SMARTS patterns, perform filtering in multiprocessing mode on all available CPUs without loading all data into memory, and write out a SMILES file, type:

```
% RDKitFilterPAINS.py --mp yes -i Sample.smi -o SampleOut.smi
```

To count the number of molecules not containing any substructure corresponding to PAINS SMARTS patterns, perform filtering in multiprocessing mode on all available CPUs by loading all data into memory, and write out a SD file, type:

```
% RDKitFilterPAINS.py --mp yes --mpParams "inputDataMode,InMemory"
-i Sample.smi -o SampleOut.sdf
```

To count the number of molecules not containing any substructure corresponding to PAINS SMARTS patterns, perform filtering in multiprocessing mode on specific number of CPUs and chunk size without loading all data into memory, and write out a SD file, type:

```
% RDKitFilterPAINS.py --mp yes --mpParams "inputDataMode,Lazy,
numProcesses,4,chunkSize,8" -i Sample.smi -o SampleOut.sdf
```

To only count the number of molecules not containing any substructure corresponding to PAINS SMARTS patterns without writing out any file, type:

```
% RDKitFilterPAINS.py -m count -i Sample.sdf -o SampleOut.smi
```

To count the number of molecules containing any substructure corresponding to PAINS SMARTS patterns and write out a SD file with computed 2D coordinates, type:

```
% RDKitFilterPAINS.py -n yes -i Sample.smi -o SampleOut.sdf
```

To count the number of molecules not containing any substructure corresponding to PAINS SMARTS patterns family of Type A in a CSV SMILES file and write out a SD file, type:

```
% RDKitFilterPAINS.py --painsMode A --infileParams  
  "smilesDelimiter,comma,smilesTitleLine,yes,smilesColumn,1,  
  smilesNameColumn,2" --outfileParams "compute2DCoords,yes"  
  -i SampleSMILES.csv -o SampleOut.sdf
```

AUTHOR

Manish Sud(msud@san.rr.com)

SEE ALSO

RDKitFilterChEMBLAlerts.py, RDKitConvertFileFormat.py, RDKitSearchSMARTS.py

COPYRIGHT

Copyright (C) 2026 Manish Sud. All rights reserved.

The functionality available in this script is implemented using RDKit, an open source toolkit for cheminformatics developed by Greg Landrum.

This file is part of MayaChemTools.

MayaChemTools is free software; you can redistribute it and/or modify it under the terms of the GNU Lesser General Public License as published by the Free Software Foundation; either version 3 of the License, or (at your option) any later version.